

Practical 06: Genetic Algorithms for 0/1 Knapsack

SE3062 Intelligent Systems | Name: _____ | Student ID: IT24100532

1. Problem summary

In the 0/1 knapsack problem, we have n items, each with a value v_i and a weight w_i , and a bag with capacity C . We pick items to **get the highest total value without going over the capacity**. Each item is either taken (1) or left (0).

I used two standard instances from Pisinger's smallcoeff set (uncorrelated items). The files include the known best answer, so I can see how close the GA gets.

Instance	Items (n)	Capacity (C)	Known optimum
knapPI_1_100_1000_1 (main)	100	995	9147
knapPI_1_500_1000_1 (part C)	500	2543	28857

2. GA design

- **Library:** DEAP 1.4.4 (Python). All experiments are in one script, `knapsack_ga.py`.
- **Encoding:** a list of n bits. 1 = take the item.
- **Fitness (hard penalty):** $\text{value}(x) - M \cdot \max(0, \text{weight}(x) - C)$ with $M = 10 \cdot \max(v)$. A bag that is too heavy gets a big negative score, so only valid bags (fitness ≥ 0) can score well.
- **Baseline:** tournament selection ($k = 3$), 2-point crossover ($p_c = 0.9$), bit-flip mutation ($p_m = 0.02$ per gene), elitism $E = 2$, 200 generations, population 150.
- **Fair runs:** each variant changes **one** setting from the baseline and runs **10 times with seeds 0–9**. I report the mean $\pm$ standard deviation of the final best fitness.
- **Convergence speed:** the generation where a run's best fitness first reaches the baseline's median final best (8267.5). I also record when the first valid bag appears.

DEAP's `eaSimple` has no elitism, so I wrote a short generation loop using DEAP's operators. DEAP has no rank selection, and its roulette needs positive fitness, so I added both (roulette shifts all fitness values up so the worst one has weight 1).

3. Results

3.1. A. Baseline

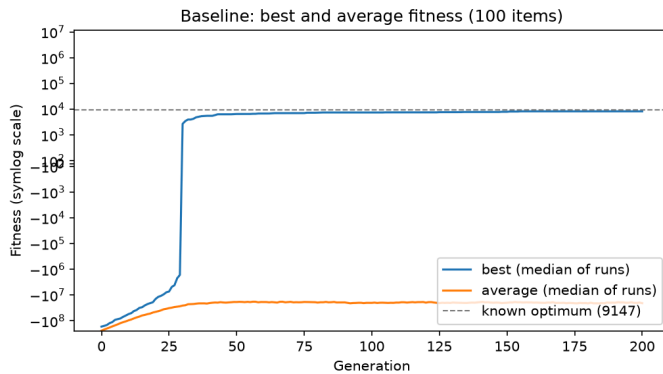


Figure 1: Baseline: median best and average fitness over 10 runs (symlog scale, negative = overweight).



Figure 2: Output of the full experiment run.

The first valid bag appears around generation 30. After that, the best fitness climbs to **8293.1 $\pm$ 386.1** (9.3% below the optimum). The average stays around -18 million all the time: every generation, mutation and crossover produce some overweight children, and their huge penalty drags the average down.

3.2. B. Operator comparison (100 items)

Variant	Best fitness (mean $\pm$ sd)	Gens to reach baseline median	First valid gen	Notes
Baseline ($k=3$, 2pt, $p_m=0.02$)	8293.1 $\pm$ 386.1	146 (5/10 runs)	30	reference
Tournament $k=2$	7180.9 $\pm$ 462.7	not reached	66	less pressure
Tournament $k=4$	8916.0 $\pm$ 198.5	81.5 (10/10)	27	more pressure
Roulette	5277.4 $\pm$ 666.7	not reached	120	fitness-proportional
Rank	7035.1 $\pm$ 786.8	not reached	56	rank-proportional
1-point crossover	8442.3 $\pm$ 536.2	108 (7/10)	32	fewer cut points
Uniform crossover	8665.3 $\pm$ 339.1	126 (9/10)	24.5	diversity trade-off
$p_m = 0.01$	9109.5 $\pm$ 80.4	55 (10/10)	25.5	less mutation
$p_m = 0.05$	no valid bag in 9/10 runs	not reached	148 (1 run)	too much mutation
$p_m = 0.10$	no valid bag in any run	not reached	never	risk of randomization

Table 1: Mean $\pm$ sd of the final best fitness over 10 runs (seeds 0–9). “Gens to reach baseline median” is the median over the runs that reached 8267.5.

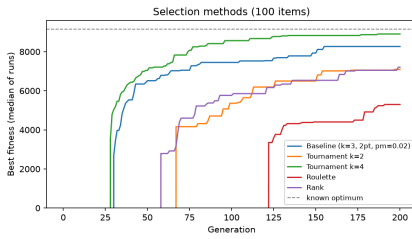


Figure 3: Selection.

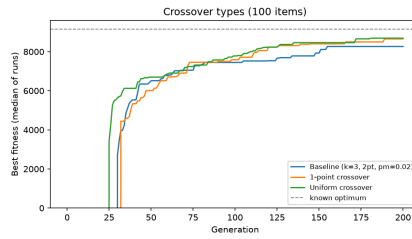


Figure 4: Crossover.

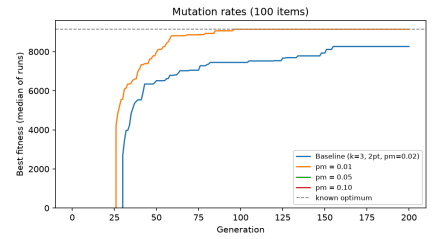


Figure 5: Mutation rate.

3.3. C. Larger instance (500 items)

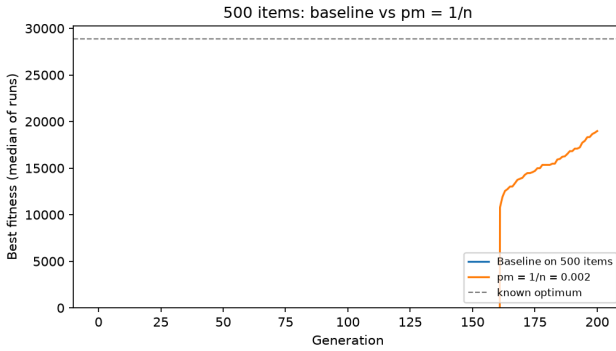


Figure 6: 500 items: median best fitness over 10 runs.

Setting	Best fitness	First valid gen
Baseline ($p_m = 0.02$)	no valid bag (0/10)	never
$p_m = 1/n = 0.002$	18938.7 $\pm$ 1135.9	160

With 500 items, $p_m = 0.02$ flips about 10 genes in every child, so the GA never gets under the capacity. Scaling mutation to $1/n$ (about 1 flip per child) fixes this. It still needs 160 generations to find a valid bag, and it is 34% below the optimum at generation 200 because it is still climbing.

4. Discussion

Pressure vs. diversity. Stronger selection pressure worked better here. Tournament $k = 4$ reached the baseline’s median in about 82 generations and ended within 2.5% of the optimum. $k = 2$, rank and roulette are weaker: they let poor bags survive, so the search wanders. Roulette was the worst (5277). Because of the hard penalty, overweight bags score millions below zero while valid bags score 0 to 9147. After shifting, this huge gap makes all valid bags look almost equally good, so the best ones get almost no extra chance of being picked. With a small population and only 200 generations, keeping good bags matters more than keeping variety.

Crossover. Uniform crossover (8665) did better than 2-point (8293) and 1-point (8442). It mixes genes from anywhere in the parents, which adds diversity and helps find good item combinations. It reached the target in 9 of 10 runs, against 5 of 10 for the baseline.

Mutation. This was the biggest effect. p_m is per gene, so the expected number of flips per child is $n \cdot p_m$. With $p_m = 0.01$ (1 flip), the GA found the exact optimum 9147 in 8 of 10 runs (9109.5 $\pm$ 80.4). $p_m = 0.05$ and 0.10 (5–10 flips) are close to a random search. Almost every child becomes overweight, so the GA cannot keep valid bags. Part C shows the same thing, so p_m should be scaled to the problem size (about $1/n$).

Convergence. All good runs follow the same shape: a long phase with no valid bag, a jump when the first valid bag appears, then slow improvement until progress stops. The hard penalty makes the first phase long because a random starting bag takes about half the items and is about $25\times$ over the capacity. A smarter start (for example, random bags that fit, or repairing overweight bags) would make the GA converge much faster.

Reproducibility: seeds 0–9 for every variant. Run `python knapsack_ga.py` (about 1.5 min) to regenerate all tables and plots.